

慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/05/07)

05 - 言語モデルで文章を統計的に扱う

情報工学科 准教授 高道 慎之介



Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習 1	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	
第13回	2026/07/09	自然言語処理から音声言語処理へ	おまけ
第14回	2026/07/16	演習 2	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：単語列の確率，ニューラルネットワーク

単語列で考えてみよう

私 / は / リンゴ / を / 食べる

彼 / は / バナナ / が / 好き / だ

私 / は / 公園 / に / 行く

リンゴ / が / 落ちる

私 / は / 赤い / リンゴ / を / 買う

空 / は / 青い

私 / は / 彼 / と / リンゴ / を / 分けた

彼女 / は / メロン / を / 食べる

私 / は / リンゴ / が / 大好き / だ

犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私, は}) = \underbrace{P(\text{は}|\text{私})}_{\text{“私” の後に “は” が出現する確率}} \underbrace{P(\text{私})}_{\text{“私” の出現確率}}$$

“私” の後に “は” が “私” の出現確率
出現する確率

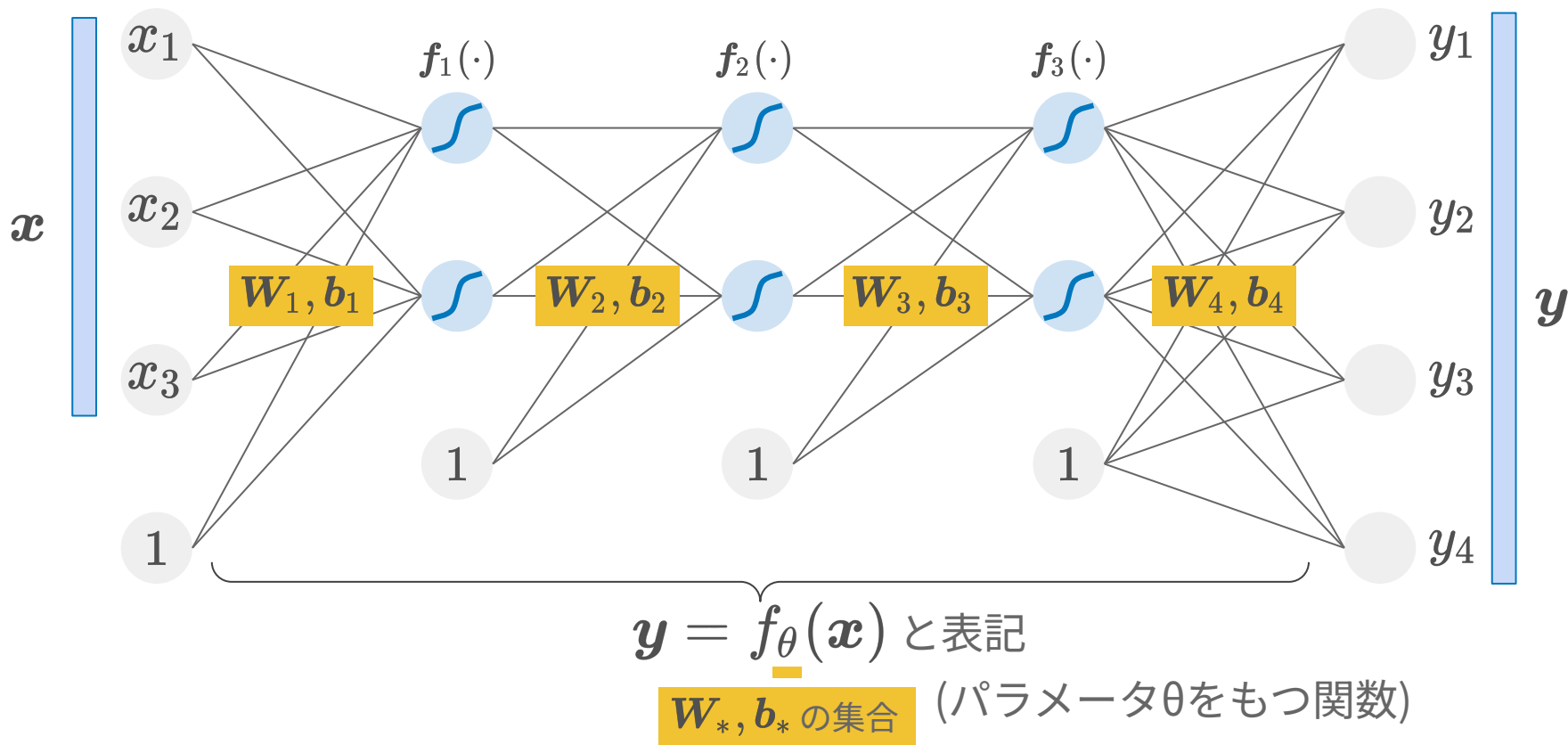
$$P(\text{私}) = \frac{5}{50} \quad (50 \text{ 単語のうち } 5 \text{ 回出現})$$

$$P(\text{は}|\text{私}) = \frac{5}{5} \quad (\text{“私” に後続する } 5 \text{ 単語のうち } 5 \text{ 回出現})$$

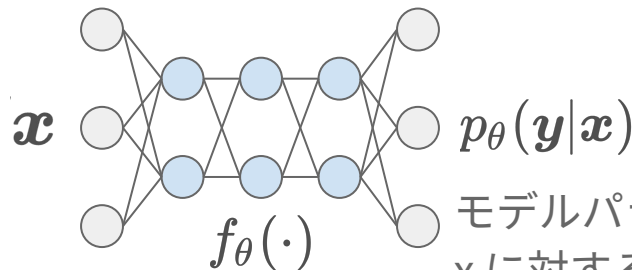
連続 3 単語以上にも拡張可能

$$P(\text{私, は, 彼}) = P(\text{彼}|\text{は, 私})P(\text{は}|\text{私})P(\text{私})$$

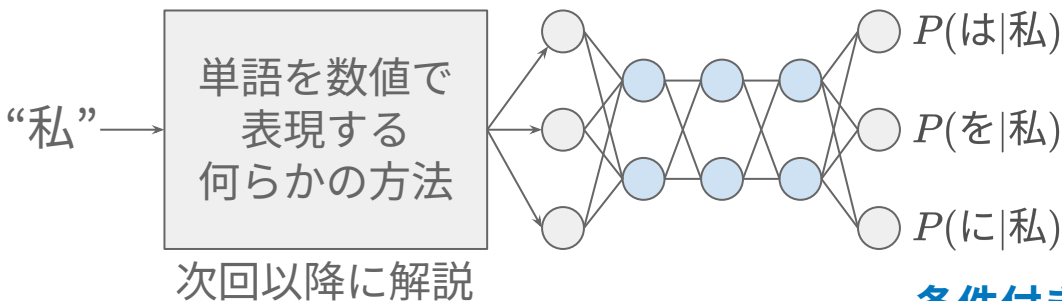
多層パーセプトロン



ニューラルネットワークを 離散確率変数の確率分布を出力するモデルだと考える



モデルパラメータ θ で計算される,
 x に対する y の条件付き確率



単語列で考えてみよう

私 / は / リンゴ / を / 食べる
彼 / は / バナナ / が / 好き / だ
私 / は / 公園 / に / 行く
リンゴ / が / 落ちる
私 / は / 赤い / リンゴ / を / 買う
空 / は / 青い
私 / は / 彼 / と / リンゴ / を / 分けた
彼女 / は / メロン / を / 食べる
私 / は / リンゴ / が / 大好き / だ
犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私}, \text{は}) = P(\text{は}|\text{私})P(\text{私})$$

“私” の後に “は” が “私” の出現確率
出現する確率

$$P(\text{私}) = \frac{5}{50} \quad (\text{50 単語のうち 5 回出現})$$

$$P(\text{は}|\text{私}) = \frac{5}{5} \quad (\text{“私” に後続する 5 単語のうち 5 回出現})$$

連続 3 単語以上にも拡張可能

$$P(\text{私}, \text{は}, \text{彼}) = P(\text{彼}|\text{は}, \text{私})P(\text{は}|\text{私})P(\text{私})$$

条件付き確率を計算する,
パラメータ θ を持つニューラルネット

分布仮説 (distributional hypothesis)

“同じ文脈で現れる単語は同じ意味を持つ傾向にある” [Harris, 1954]

単語が持つ意味は、**周りの環境 (共起語)** を用いて説明できる

I saw **a** **girl** **with** red shoes

I saw **a** **lady** **with** red shoes

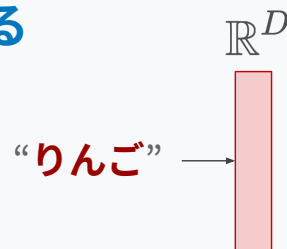
{girl, lady} は似た意味の単語

1つ前に **a**、1つ後に **with** が共起する
という同じ環境を持つため

代わりに、密な実数値ベクトルで表すことを考える → 単語埋め込み (word embedding)

あらかじめ定めた次元数 D の密ベクトル (=埋め込み) に写像する

- 単語の集合 (語彙): $V = \{w_1, \dots, w_i, \dots, w_{|V|}\}$
- 写像: $f: V \rightarrow \mathbb{R}^D$



単語埋め込み行列による写像 (lookup)

実質的に行列の特定行の抽出. いわゆる Word Embedding 層.

$$\begin{bmatrix} -0.1 \\ 3.0 \end{bmatrix}_{\mathbb{R}^D} = \begin{bmatrix} -0.1 & 0.2 & 0.3 \\ 3.0 & 2.0 & 1.0 \end{bmatrix}_{\mathbb{R}^{D \times |V|}} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}_{\text{one-hot}} \quad \text{"りんご"}$$

* one-hot ベクトルの次元数爆発は、この時点では解決できていない

LLM 時代においては、サブワード単位が主流

基本的な考え方：頻度の高い単語を分割すること

the companies are expanding

the **compan _ies** are **expand _ing**

サブワード化のメリット

学習パラメータの共有

複合語の効率化

- 通常銃猟 → 通常 _銃猟
- 緊急銃猟 → 緊急 _銃猟

活用語の効率化

- 美しい → 美し _い
- 美しく → 美し _く

計算コストの削減と未知語の解消

- 単語単位よりも語彙数を大幅に削減
- 未知語も部分単語に分解するため、未知の入力が存在しなくなる。

今日の目的：

文章を統計的に扱う言語モデルとは何か

言語モデルとは

Language model

言語モデル：単語列から次の単語を予測する統計モデル

$$P(w_{N+1} | w_1, w_2, \dots, w_N)$$

次の単語を出力する確率 これまでの文脈 N 単語が与えられている

慶應義塾大学 理工学部 情報工学科 (Department of Information and Computer Science) は、最先端のコンピュータ技術から、それらを社会に役立てるための知能システムまでを幅広くカバーする **???**

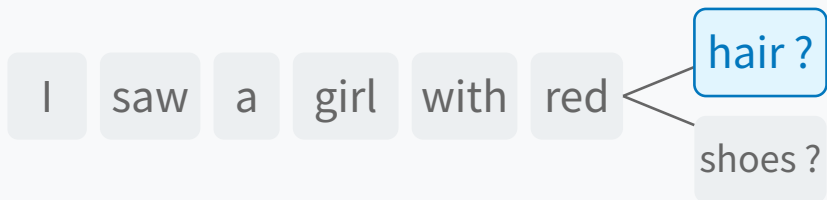
単語予測を繰り返せば文章を生成できる。

- (LLMでは単語ではなくサブワードを扱うが簡単のため単語と表記．扱いは同様)

* 簡単のため「次の」と表記しているが、「前の」や「間の」単語の確率を予測するものも、同様に言語モデルと呼ぶ．次の単語を予測する言語モデルは、自己回帰型言語モデルと呼ばれる。

どんなことに使える？

文章を生成する



テキスト→テキスト変換



「言語らしさ」を測る



X → テキスト変換



大規模言語モデルも言語モデルの一種 (詳細は後日の講義にて)

学習

大量のテキストデータを用意する

私 / は / リンゴ / を / 食べる
彼 / は / バナナ / が / 好き / だ
私 / は / 公園 / に / 行く
...

私 / は / 赤い / リンゴ / を / 買う
空 / は / 青い
私 / は / 彼 / と / リンゴ / を / 分けた
...

* 実際にはサブワード単位に分割される

推論

$w_1, w_2, \dots \longrightarrow$ **大規模言語モデル (LLM)** $\longrightarrow P(w_{N+1} | w_1, w_2, \dots, w_N)$

n-gram 言語モデル

n-gram language model

ニューラルネットワーク以前の方法 (現在でも使用する場合がある).
KenLM (<https://github.com/kpu/kenlm>) を使うと実装が楽.

第 02 回の課題が(文字) n-gram 言語モデル

文字 n-gram (連続する n 文字) の確率を計算せよ

data.txt

いあいあういいういいうあうあいういいうし
あういああああいういいうあいうあうあいう
いああうあいうあいういういいういあいうう
うああういううあいういいういあいういああ

```
1 # 課題： data.txt を読み込んで、文字の n-gram (n = 1, 2) を
2 # 表示するプログラムを作成してください。
3 # すでに書かれている箇所は変更しても構いません。
4
5 def n_gram(text, n):
6     """テキストを n-gram に分割する関数"""
7
8
9 if __name__ == "__main__":
10     """メイン関数"""
11
12     unigram = n_gram(text, 1)
13     bigram = n_gram(text, 2)
14
15     # 1-gram (unigram, ユニグラム)
16     print("あ", unigram["あ"])
17     print("い", unigram["い"])
18     print("う", unigram["う"])
19
20     # 2-gram (bigram, バイグラム)
21     print("あ あ", bigram["あ あ"])
22     print("あ い", bigram["あ い"])
23     print("あ う", bigram["あ う"])
24     print("い あ", bigram["い あ"])
```

“あ”が出現する確率
（“あ”の出現回数 / 全文字数）

“あ”と“あ”が連続
して出現する確率

(単語) n-gram モデル

直前の N - 1 単語に依存して、当該単語の確率を計算する確率モデル.

- それ以前の単語は無視する

$$P(w_n | w_1, w_2, \dots) = P(w_n | w_{n-N+1}, \dots, w_{n-1})$$

私	が	麵	を	食べ	る
w_1	w_2	w_3	w_4	w_5	w_6

1-gram (ユニグラム)	$P(w_6)$
2-gram (バイグラム)	$P(w_6 w_5)$
3-gram (トライグラム)	$P(w_6 w_4, w_5)$
4-gram (フォーグラム)	$P(w_6 w_3, w_4, w_5)$

パラメータ学習法 (最尤推定法)

学習データにおける単語列の出現回数をカウントすればよい。

N 単語 の出現回数

$$P(w_n | w_{n+N+1}, \dots, w_{n-1}) = \frac{\text{Count}(w_{n+N+1}, \dots, w_{n-1}, w_n)}{\text{Count}(w_{n+N+1}, \dots, w_{n-1})}$$

直前の N-1 単語 (文脈) の出現回数

例えば 2-gram を考えると…

私 / は / リンゴ / を / 食べる
彼 / は / バナナ / が / 好き / だ
私 / は / 公園 / に / 行く
リンゴ / が / 落ちる
私 / は / 赤い / リンゴ / を / 買う
空 / は / 青い
私 / は / 彼 / と / リンゴ / を / 分けた
彼女 / は / メロン / を / 食べる
私 / は / リンゴ / が / 大好き / だ
犬 / が / 走って / いる

$$P(\text{は} | \text{私}) = \frac{\text{Count}(\text{私}, \text{は})}{\text{Count}(\text{私})} = \frac{5}{5}$$

$$P(\text{彼} | \text{は}) = \frac{\text{Count}(\text{は}, \text{彼})}{\text{Count}(\text{は})} = \frac{1}{8}$$

n-gram 言語モデルの利点と限界

⊕ 利点

1. 確率計算が簡便・直感的

- 学習：出現数のカウントのみ
- 推論：確率値を読み出すのみ

2. 小規模データに強い

- 専門用語が多い特定分野や、リソースの少ない希少言語で有効

⊖ 限界・欠点

1. パラメータの指数的増加

- 語彙数 $|V|$ に対し $|V|^N$ の計算量
- 大きい n (例: $n > 4$) で計算が困難．
長的文脈を保持できない

2. ゼロ頻度問題

- 未学習の単語列の確率は一律 0
- スムージングでの補完は対症療法に過ぎず、本質的解決ではない

n-gram 言語モデルの限界

限界 1：関係単語を扱えない



語彙的関係の定義 (シソーラス、オントロジー辞書に収録)

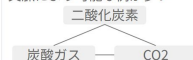
同義関係

定義：
(ほとんど) 全ての文脈で
置き換え可能な単語

具体例：

- 車、自動車
- 二酸化炭素、炭酸ガス、CO2

※完全な互換性なくても、
文脈により可能な例が多い



東京大学 宮尾先生「自然言語処理」スライドより引用

上位・下位関係

定義：
単語Aの全性質をBが持つなら、
AはBの上位語 (B is a A)

具体例：

- 哺乳類 > 犬、猫
- 家具 > テーブル

※Aの集合がBを包含する関係。
条件付でBをAに置換可能



全体・部分関係

定義：
物理的に含む・含まれる
関係 (Has-a関係)

具体例：

- 顔 > 目、鼻、口
- 自動車 > エンジン

※上位語との違い：
赤いイヌ → 赤い動物 (○)
赤い鼻 ≠ 赤い顔 (×)



第03回スライドから引用

限界 2：離れた単語を扱えない

美味しい とは 全く 思わ ない

否定：美味しいの否定が離れている

鍵 を 失くして それ を 探した

照応：代名詞の指す内容が離れている

→ 解決策：skip-based モデル

→ 解決策：class-based n-gram モデル

単語の出現頻度は偏る

頻出単語と稀発単語

● 日本語の例

頻発：て, に, を, は

稀発：やんごとない, 猪口才な

● 英語の例

頻発：the, a, of

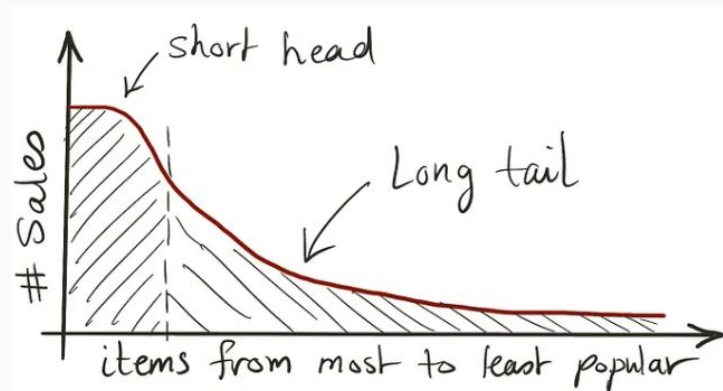
稀発

：pneumonoultramicroscopicsilicovolcanoconiosis (超微視的珪質火山塵肺疾患)

ロングテール分布

超大規模データでも稀発単語は多数存在し、分布は**ロングテール**となる。

n-gram (単語列) ではこの傾向がより顕著に現れる。

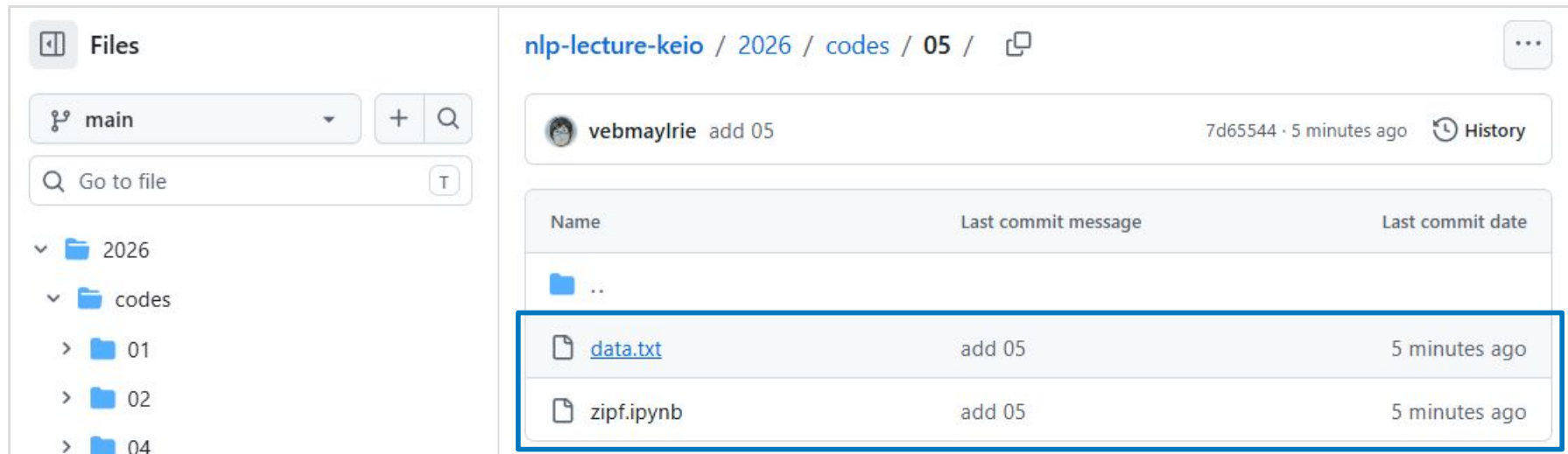


<https://medium.com/codex/machine-learning-the-long-tail-paradox-1cc647d4ba4b>

Zipf 則 (ジップ則)

データ集合のある要素の出現頻度が k 番目に大きいとき, その頻度は最頻要素の頻度の $1/k$ である. 単語もこれに従うことが知られている.

- <https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/05>



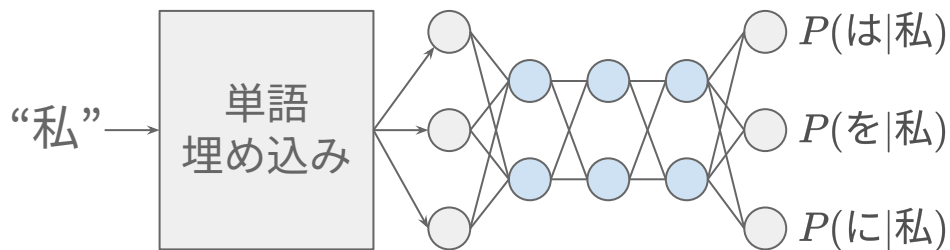
The screenshot displays the GitHub interface for the repository 'nlp-lecture-keio'. The left sidebar shows the file structure, with the '2026' folder expanded, revealing the 'codes' folder and its subfolders '01', '02', and '04'. The main area shows the commit history for the '05' directory. A table lists the files added in the last commit, which was made 5 minutes ago by user 'vebmaylrrie'.

Name	Last commit message	Last commit date
..		
data.txt	add 05	5 minutes ago
zipf.ipynb	add 05	5 minutes ago

初期のニューラル言語モデル

Neural language model in early era

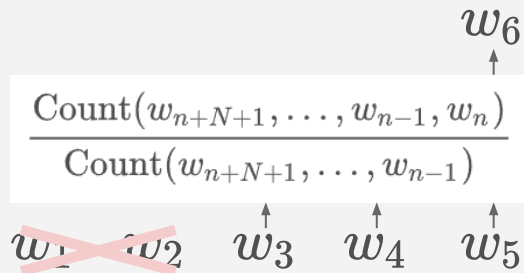
2013～2017年ごろのニューラルネットワーク × NLP



n-gram 言語モデルからニューラル言語モデルへ

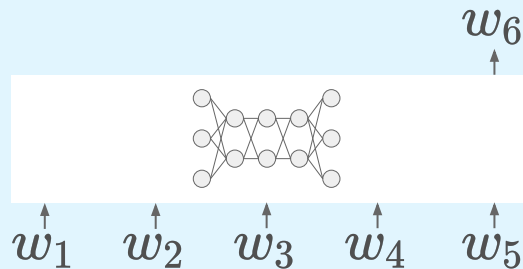
n-gram 言語モデル

- 過去の (N) 単語から、当該単語の出現確率を求める。
- パラメータ数は $|V|^N$
- N を大きくしづらいため、長い文脈を扱えない**

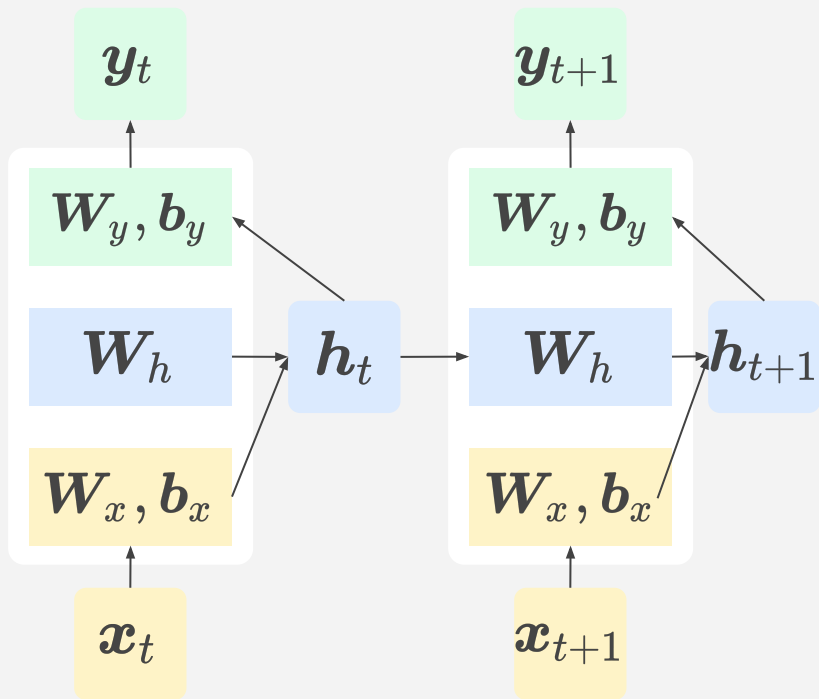


ニューラル言語モデル

- 過去の単語から、当該単語の出現頻度を出力するニューラルネットワーク (NN) を用いる。
- パラメータ数 = NN のパラメータ数
- 少ないパラメータ数で N を大きくできる (=長い文脈を扱える)**



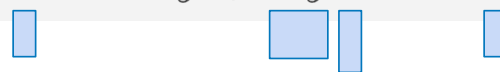
準備：リカレントニューラルネットワーク (RNN)



$$h_t = f_h (W_h h_{t-1} + W_x x_t + b_x)$$



$$y_t = f_y (W_y h_t + b_y)$$



- 1つ前の時刻の内部状態 h を使う
- 以下の3つからなる
 - 入力 x に作用する項
 - **前の時刻の h に作用する項**
 - 出力 y を計算する項

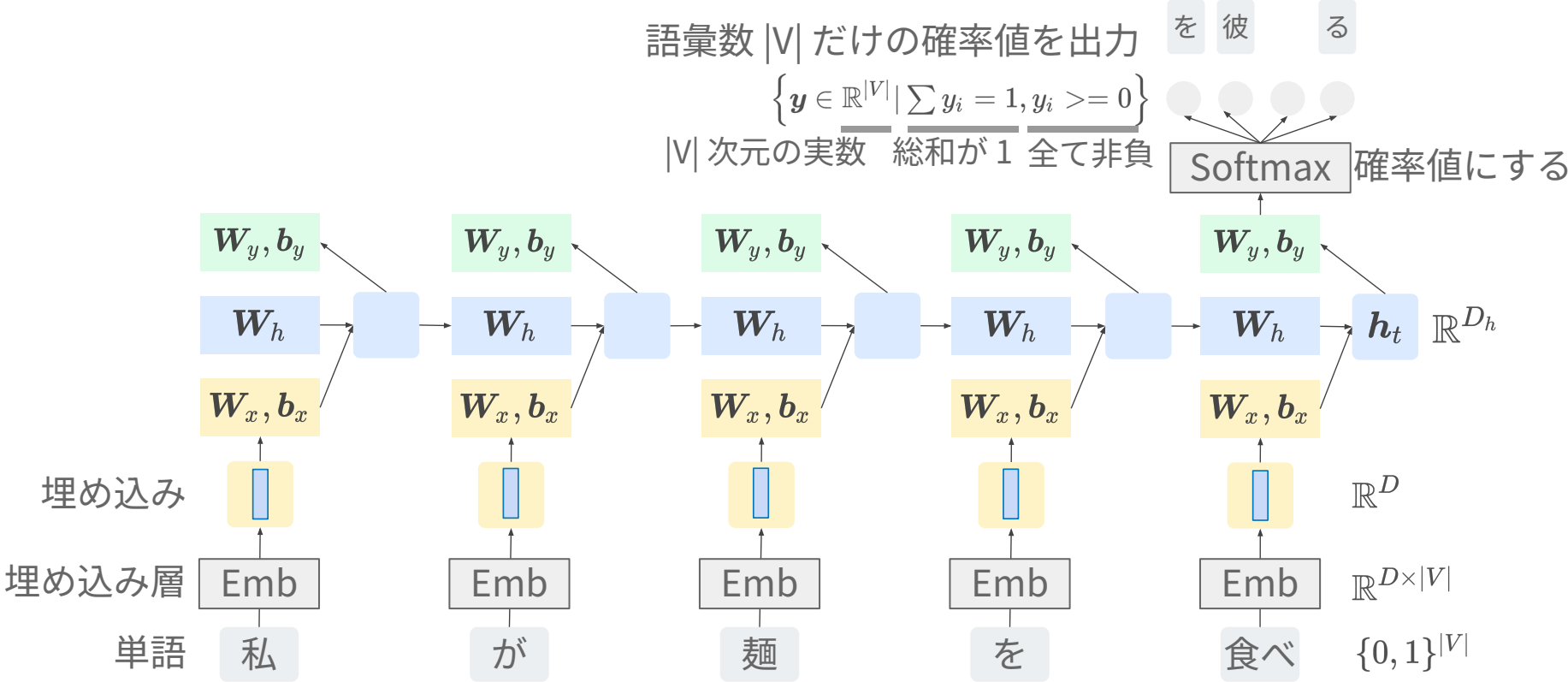


W_x, b_x

W_h

W_y, b_y

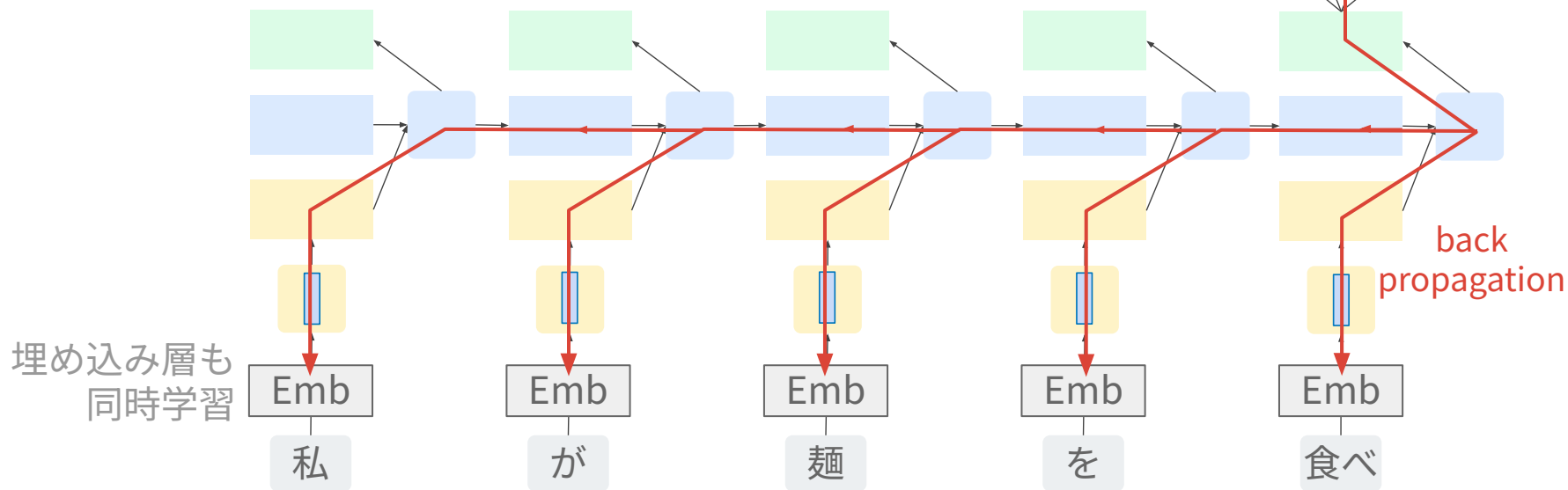
RNN 言語モデル (RNNLM)



RNN 言語モデル (RNNLM) の学習

学習データを用いると次の単語の正解が分かる。
そこからバックプロパゲーションにより
過去の全ての単語位置で学習が起こる。

負例 を 彼 る 正例
softmax cross entropy



RNNLM を実践してみよう

<https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/05>

☰

 ... / nlp-lecture-keio

+

🕒

🔗

📄

📧



Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

📁 Files

🔑 main

+

🔍

▼

📁 2026

▼

📁 codes

➤

📁 01

➤

📁 02

➤

📁 04

▼

📁 05

nlp-lecture-keio / 2026 / codes / 05 / 📄

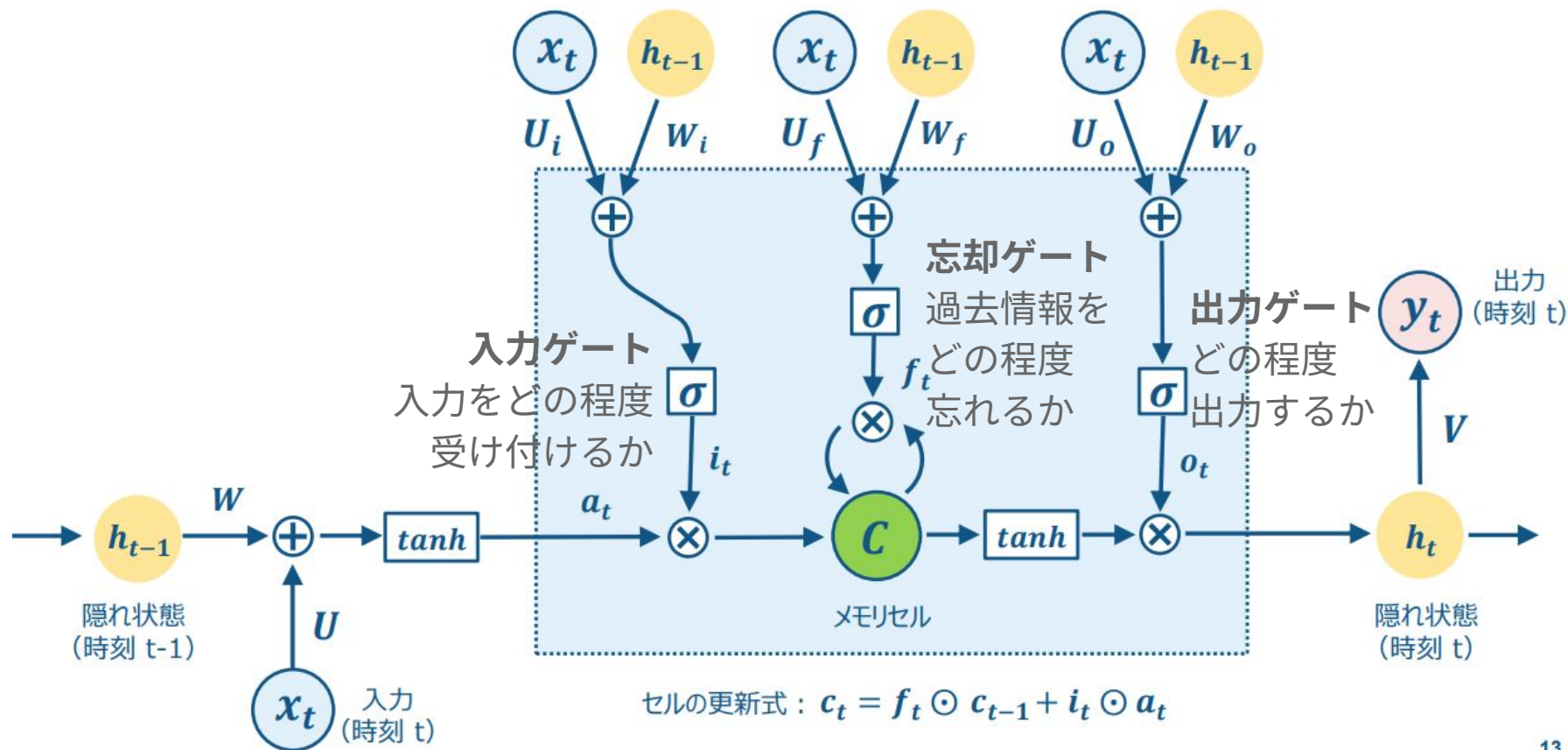
...

 vebmaylrie add .to(device) ●

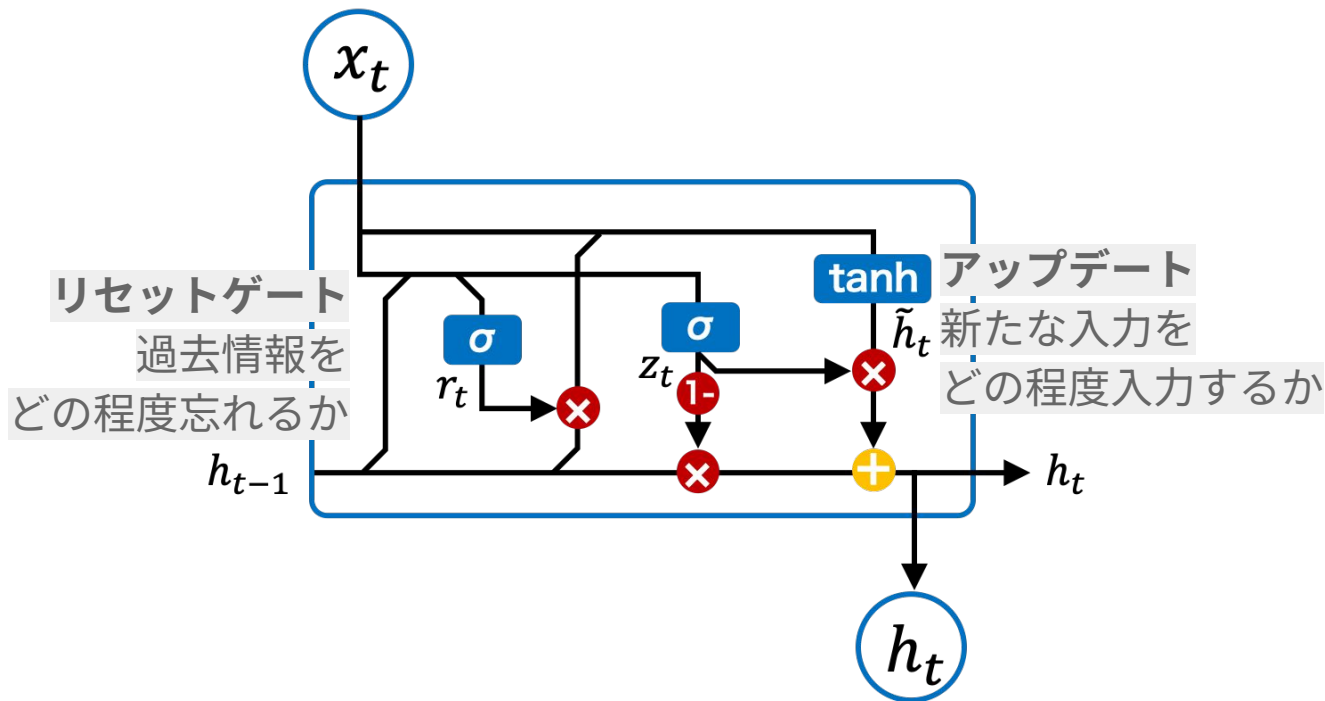
58cf67d · 1 minute ago 🕒 History

Name	Last commit message	Last commit date
📁 ..		
📄 data-rnnlm.txt	add rnnlm	11 minutes ago
📄 data.txt	add 05	last week
📄 rnn-lm.ipynb	add .to(device)	1 minute ago

LSTM : RNN の発展形



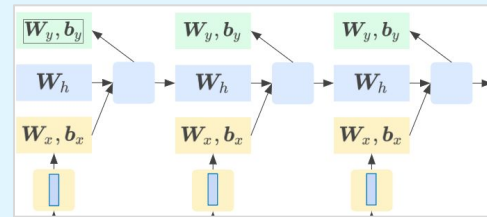
GRU : LSTMをよりシンプルにしたバージョン



RNN LM は n-gram LMの弱点を本当に克服した？

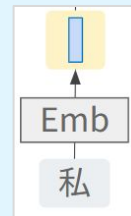
長い文脈 (“理論上は” 無限) を捉えられるようになった！

- 学習時：勾配が過去の全ての単語に勾配を伝える
- 推論時：過去の全ての単語から、ベクトル h を計算



未知の単語列にも対応するようになった！

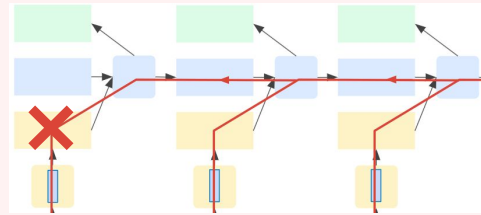
- 単語埋め込みにより、意味的に類似した単語同士を、近いベクトルとして表現できるようになったため。



RNN LM の欠点 (LSTM, GRU も同様)

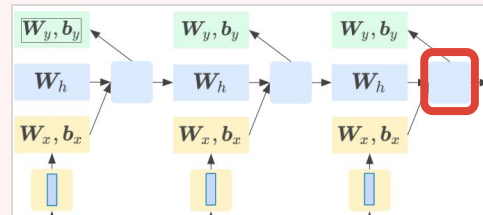
勾配が消失して過去の単語に伝播しない

- 理論上は無限だが実際には数十単語まで遡ると勾配が消失する



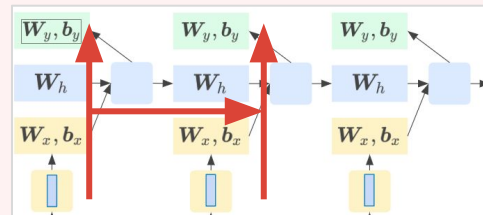
過去の単語の文脈が過度に圧縮される

- 過去のすべての単語の情報が、固定長のベクトルで表現されるため、遠い過去の情報が消失する



並列計算できず計算効率が悪い

- RNN は過去の単語について全て推論しないと、今の単語について推論できない



Transformer LM に続く道 (詳細は後日)

勾配が消失して過去の単語に伝播しない

- 理論上は無限だが実際には数十単語まで遡ると勾配が消失する

注意機構 (attention) により
必要な単語だけに勾配を強く！

過去の単語の文脈が過度に圧縮される

- 過去のすべての単語の情報が、固定長のベクトルで表現されるため、遠い過去の情報が消失する

注意機構 (attention) により
必要な単語の情報だけを圧縮！

並列計算できず計算効率が悪い

- RNN は過去の単語について全て推論しないと今の単語について推論できない

自己注意機構 (self attention) により
並列計算可能！

言語モデルを評価する

Evaluating language models

言語モデルの“良さ”をどのように定量化する？
(生成した文章の良さは後日)

Negative log-likelihood (NLL, 負の対数尤度)

文 x の単語数 *サブワードを単位とするならサブワード数

$$\text{NLL} = - \sum_{x \in \mathcal{X}_{\text{test}}} \log P(x) = - \sum_{x \in \mathcal{X}_{\text{test}}} \sum_{t=1}^{|x|} \log P(w_t | w_{<t})$$

評価セットの文 x で総和 文 x の尤度 過去単語から次の正解単語を出力する確率

- **尤度が大きい（NLLが小さい） = その文を高い確率で出力できる**

- 例えば、当該単語のみ確率が 1 なら、 $-\log P(w) = 0$ で最小
- モデル学習時に最小化される目的関数（クロスエントロピー）と同じ

*後の講義で触れる強化学習の目的関数とは異なる

- 余談

- なぜ「負」？ → DNNの目的関数のように「小さければよい」にするため
- なぜ「対数」？ → 計算機のアンダーフローを避けるためなど

エントロピー (entropy)

$$H = - \frac{1}{\sum_{x \in \mathcal{X}_{\text{test}}} |x|} \sum_{x \in \mathcal{X}_{\text{test}}} \sum_{t=1}^{|x|} \log P(w_t | w_{<t})$$

単語数の総和で正規化 文 x の単語数 過去単語から次の正解単語を出力する確率

- **エントロピーが小さい = 確率分布の不確かさが小さい**
 - NLL を総単語数で正規化 → 負の対数尤度の単語平均

パープレキシティ (perplexity)

$$\text{Perplexity} = e^{\overline{H}^{\text{エントロピー}}}$$

- **パープレキシティが小さい = 予測時の候補を少なく絞れている**
 - エントロピーの指数 → 平均確率の逆数 → 予測時の候補数
 - Perplexity = 1 : 全単語を確率 1 で予測可能
 - Perplexity = 10 : 平均的には 10 択から生成している
 - Perplexity = $|V|$: 全語彙からランダムに選んでいること (に近い)
- 言語モデルの性能を直感的に理解しやすい尺度

言語モデルを評価してみよう (RNNLM と同じファイル)

<https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/05>

The screenshot shows the GitHub interface for the repository `nlp-lecture-keio`. The left sidebar displays the file tree with the path `2026 / codes / 05` selected. The main content area shows the commit history for this directory, with three commits highlighted by blue boxes:

Name	Last commit message	Last commit date
..		
data-rnnlm.txt	add rnnlm	11 minutes ago
data.txt	add 05	last week
rnn-lm.ipynb	add .to(device)	1 minute ago

まとめ

Conclusion

まとめ

- 単語列から (次の) 単語を予測する統計モデル
- n-gram 言語モデル
 - 単語の出現回数をカウントするモデル
- RNN 言語モデル
 - RNN を用いるニューラル言語モデル
- 言語モデルの評価
 - NLL, エントロピー, パープレキシティ

課題 (どれか 1 つを選択し提出せよ)

1. Zipf 則の確認

- 大学に提出した日本語レポートを準備し単語の出現頻度を学年毎に集計する
- 出現頻度が学年ごとにどのように変化しているか考察せよ (例: Zipf 則, 語彙数, ロングテール, etc.)
- 文の分割は句点・疑問符・感嘆符などに基づいて規則的に行うこと
- 単語分割は MeCab (前回講義のプログラム参照) を使用すること

2. RNN言語モデルの学習

- 評価データセットを固定せよ.
- 学習データセットあるいは言語モデルを工夫し, 評価データセットに対するパープレキシティを改善させよ.